

Replit Agent Prompt — Atlas Wave 4a: Notes + Project Enhancement + Infrastructure

Read this entire prompt before taking any action.

1. Overview

Wave 4 introduces the second major module after Tasks: **Notes**. It also enhances the existing Project entity to serve as the universal identity for any sustained initiative — Projects, Goals, Habits — through a Type field. And it adds two pieces of infrastructure that future waves depend on: the Locale setting and the scheduled job runner.

This wave is split into three sub-waves. **This prompt covers Wave 4a only.**

- **Wave 4a (this wave): Notes + Project enhancement + infrastructure** — 4-5 weeks
- **Wave 4b (later): Tables** — 2-3 weeks
- **Wave 4c (later): Project view richness + advanced Notes/Tables features** — 2-3 weeks

The architectural insight driving this wave: a Goal isn't a separate entity type. It's a Project with `type = goal`. A Habit is a Project with `type = habit`. The Project entity becomes the binding identity for any sustained initiative, with Notes, Tasks, and (later) Tables, Calendar events, Journal entries all hanging off it. This composition approach avoids module bloat while making the system more expressive.

The work in Wave 4a:

1. **Notes module** — entity, folder hierarchy, full WYSIWYG editor, Purpose field, cross-module references, Drive sync
2. **Project enhancement** — Type field (Project, Goal, Habit), status, target_date, designated brief note
3. **Locale setting** — Settings → Preferences with currency, date format, number format, time format (presets + custom)
4. **Scheduled job runner** — infrastructure for hourly Drive sync and other periodic jobs,

with Settings management UI

5. **Durable bidirectional links** — foundation for the composition pattern that Wave 4c activates

Pre-requisites:

- Wave 3a, 3a Polish, 3c, Media wave, and Refinement v2 wave all shipped and stable
- Documents-related code already cleaned up from earlier waves (placeholder removed)
- Existing reference parser from Wave 3a (handles `#tags`, `@contexts`, `[[entities]]`)
- Wave 1's Drive integration is functional

Estimated scope: 4-5 weeks of focused work.

2. Architecture overview

2.1 Notes are flat with folders

Notes follow the same organizational pattern as Tasks/Projects:

- Folders organize notes (and other folders, up to 5 levels deep)
- Notes don't nest within other notes
- Cross-references via `[[Note Title]]` create semantic connections without forcing hierarchy

This avoids the Notion confusion of "is this a page or a section?" Every note is a peer; structure emerges from folders (for organization) and references (for connection).

2.2 Notes optionally belong to a Project

A Note can be standalone (orphan) or attached to a Project via `project_id`. Standalone notes live in the Notes module's folder structure. Project-attached notes appear in both the Notes module AND in the Project's view.

This dual visibility matters: when working in the Notes module, you see all your notes (including project-attached ones). When working in a specific project, you see notes scoped to that project. The same note appears in both surfaces.

2.3 Project becomes the universal initiative identity

Currently a Project is a thin container for tasks. This wave enhances it:

- **Type field:** `project` | `goal` | `habit` — three values for now, more in Phase 2
- **Status field:** `active` | `paused` | `completed` | `abandoned`
- **Target date** (optional): for goals with deadlines
- **Brief note:** one note can be designated as the project's canonical description (the "what and why" of the initiative)

The Type drives UI labeling — Goals appear in a "Goals" sidebar group, Habits in a "Habits" group, Projects in a "Projects" group. Same underlying entity; different presentation reduces user hesitation about whether to use the entity for non-traditional initiatives.

2.4 Locale is global

A single Locale setting per user controls currency, date format, number format, and time format across the entire app. Tables (Wave 4b) will use it for currency display. Notes use it for date formatting. Existing modules (Tasks, Capture, Audit log) update to respect Locale.

2.5 Scheduled job runner is foundational

Hourly Drive sync needs a job runner. Several deferred Wave 1 tasks (session cleanup, trash retention, attachment cleanup) also need it. Building this infrastructure properly in Wave 4a unblocks future waves cleanly.

The runner uses `pg-boss` (Postgres-backed job queue, no Redis required, fits the simplicity principle). Jobs are stored in DB, processed by a background worker. Settings UI exposes job state for inspection.

2.6 Durable bidirectional links

Wave 3a's reference parser handles syntactic links (`[[Entity]]` , `#tag` , `@context`). Wave 4a adds a `Link` table that stores resolved references as durable graph edges — bidirectional, queryable, and stable across renames.

This is the foundation for Wave 4c's Project aggregated view ("show me everything related to this project"). Without it, Project views would have to scan note bodies for references on every load.

3. Detailed deliverables

3.1 Notes data model

```

model Note {
  id                String    @id @default(uuid()) @db.Uuid
  user_id           String    @db.Uuid
  user              User      @relation(fields: [user_id], references: [id])

  // Identity
  title             String    @db.VarChar(500)

  // Content (TipTap stores as ProseMirror JSON; we also store plain text for sea
  body_json         Json      // TipTap document state
  body_text         String    // Plain text extraction for search and Drive sync (.
  body_markdown     String    // Markdown serialization (used for Drive backup)

  // Organization
  folder_id         String?   @db.Uuid // Optional: folder placement in Notes modu
  folder            NotesFolder? @relation(fields: [folder_id], references: [id])

  project_id        String?   @db.Uuid // Optional: attach to a Project
  project           Project?  @relation(fields: [project_id], references: [id])

  // Purpose (templates)
  purpose           String    @default("note") // 'note' | 'meeting_note' | 'proje

  // Brief designation: when this note is the canonical brief for its project
  is_project_brief Boolean    @default(false)

  // Position within folder/project for ordering
  position          Decimal    @db.Decimal(20, 10) @default(0)

  // Drive sync state
  drive_file_id     String?   // Google Drive file ID for the synced .md
  drive_synced_at   DateTime? @db.Timestamptz
  drive_sync_error  String?

  // Standard fields
  created_at        DateTime   @default(now()) @db.Timestamptz
  updated_at        DateTime   @updatedAt @db.Timestamptz
  deleted_at        DateTime?  @db.Timestamptz

  // Relations
  attachments       Attachment[] // Polymorphic from Media wave
  outbound_links    Link[]      @relation("source_note")
  inbound_links     Link[]      @relation("target_note")

  @@index([user_id, deleted_at])
  @@index([user_id, folder_id, deleted_at])

```

```

    @@index([user_id, project_id, deleted_at])
    @@index([user_id, purpose, deleted_at])
    @@index([user_id, is_project_brief]) // partial: WHERE is_project_brief = true
}

model NotesFolder {
  id          String      @id @default(uuid()) @db.Uuid
  user_id     String      @db.Uuid
  user        User        @relation(fields: [user_id], references: [id])

  name        String
  parent_folder_id String? @db.Uuid
  parent_folder NotesFolder? @relation("FolderHierarchy", fields: [parent_folder_id], references: [id])
  child_folders NotesFolder[] @relation("FolderHierarchy")

  position     Decimal      @db.Decimal(20, 10) @default(0)

  created_at   DateTime      @default(now()) @db.Timestamptz
  updated_at   DateTime      @updatedAt @db.Timestamptz
  deleted_at   DateTime?     @db.Timestamptz

  notes        Note[]

  @@index([user_id, parent_folder_id, deleted_at])
}

```

Constraints to enforce server-side:

- A Note can have either a `folder_id` OR a `project_id`, but not both (project-attached notes don't also live in a Notes folder; they live in the project)
- Actually no — let me reconsider this. A note CAN be in a folder AND attached to a project. Folder is the Notes module organization; project is a logical attachment. They serve different purposes. Allow both.
- `is_project_brief = true` **requires** `project_id` IS NOT NULL . Only one note per project can have `is_project_brief = true` (enforce via partial unique index:
`UNIQUE(project_id) WHERE is_project_brief = true AND deleted_at IS NULL`)
- Folder depth limit of 5 levels (server-side check on create/move)

3.2 Project enhancement

Add to existing Project model:

```

model Project {

```

```

// existing fields...

// NEW in Wave 4a:
type          String    @default("project") // 'project' | 'goal' | 'habit'
status        String    @default("active")  // 'active' | 'paused' | 'complete'
target_date   DateTime? @db.Date // Optional: for goals with deadlines

// NEW relations
notes         Note[]

// existing fields continued...
}

```

The brief note relationship is implicit via `Note.is_project_brief`. To get the brief for a project: `prisma.note.findFirst({ where: { project_id: X, is_project_brief: true } })`.

Migration concerns:

- Existing projects default to `type = 'project'`, `status = 'active'`. No data loss.
- The Type field affects sidebar grouping (see 3.5) but doesn't change underlying behavior of existing projects.

3.3 Link table for durable bidirectional references

```

model Link {
  id          String    @id @default(uuid()) @db.Uuid
  user_id     String    @db.Uuid
  user        User      @relation(fields: [user_id], references: [id])

  // Source (where the link is written)
  source_type String    // 'note' | 'task' | 'project' | 'journal_entry' (future)
  source_id   String    @db.Uuid
  source_note Note?     @relation("source_note", fields: [source_id], references: [target_id])
  // Note: Prisma can only enforce one FK; source_id may point to other tables
  // Application-level integrity check on source resolution

  // Target (what's being referenced)
  target_type String    // 'note' | 'task' | 'project' | 'tag' | 'context' | 'journal_entry'
  target_id   String    @db.Uuid
  target_note Note?     @relation("target_note", fields: [target_id], references: [source_id])
  // Same caveat

  // Context within the source (for showing "this section mentions X")
  source_excerpt String? // ~200 char snippet around the reference
}

```

```

    created_at      DateTime @default(now()) @db.Timestamptz

    @@unique([source_type, source_id, target_type, target_id])
    @@index([target_type, target_id]) // For backlink queries
    @@index([source_type, source_id]) // For "what does this reference?"
    @@index([user_id])
  }

```

How links get created:

When a Note is saved, the editor's content is parsed for references (`[[Entity]]` , `#tag` , `@context` , `@person`). For each found reference:

1. Resolve the reference to an existing entity (search by name/title)
2. If resolved: create or update a `Link` row with `source_type='note'`, `source_id=note.id`, `target_type=resolved_type`, `target_id=resolved_id`
3. If not resolved: leave as unresolved syntactic reference (can become a "stub" — clicking offers to create the missing entity)

Links from Tasks (already created in Wave 3a) follow the same pattern: when a task is saved with references in description/notes, Link rows are created with `source_type='task'`.

Backlinks query:

Show all references TO a given entity:

```

const backlinks = await prisma.link.findMany({
  where: {
    target_type: 'note',
    target_id: noteId,
  },
  include: { /* resolve source entities */ }
})

```

This is fast (indexed on `target_type`, `target_id`) and stable across renames (links use IDs, not titles).

3.4 TipTap editor with markdown shortcuts

The editor is the highest-stakes piece of this wave. TipTap (built on ProseMirror) provides the foundation; configuration is what makes it feel right.

Required extensions:

- StarterKit (paragraphs, headings, lists, bold, italic, code)
- Markdown shortcuts (typing `#` becomes heading, `**bold**` becomes bold, etc.)
- Link extension with hyperlink support
- Code block with syntax highlighting (use `lowlight` for languages)
- Task list (interactive checkboxes within notes)
- Horizontal rule
- Blockquote
- Underline
- Strikethrough
- Custom: cross-module reference extension (handles `[]`, `#`, `@` triggers)

Editor behaviors:

- Slash command (`/`) opens block-type menu (paragraph, heading, list, code block, etc.)
- `[]` opens entity picker (notes, tasks, projects, contexts, people)
- `#` opens tag picker (existing tags only — Tag architecture from refinement v2 says no auto-create)
- `@` opens context/person picker
- Auto-save on debounce (~1 second after typing stops)
- Cmd+S triggers immediate save
- Cmd+B / Cmd+I / Cmd+U for inline formatting (standard)
- Markdown export available via menu (downloads .md file)

Reference picker UX:

When user types `[]`, a dropdown appears with searchable list:

`[]plan`

Atlas Phase 1 Plan	← Note
Q2 Planning Meeting	← Note
Plan team offsite	← Task
+ Create new note "plan"	

Type characters → list filters. Arrow keys navigate. Enter selects. Escape cancels. The “Create new” option at the bottom creates a stub entity (note by default) and links to it.

The picker shows entity type with icon/color so users distinguish notes from tasks from projects.

Storage format:

TipTap stores documents as ProseMirror JSON (good for round-trip fidelity). Atlas stores three representations of each note:

1. `body_json` — ProseMirror JSON (canonical; what the editor reads/writes)
2. `body_text` — Plain text extraction (for full-text search via Postgres tsvector)
3. `body_markdown` — Markdown serialization (for Drive sync)

Generate `body_text` and `body_markdown` server-side on every save. Use TipTap’s serialization helpers.

3.5 Notes module UI

3.5.1 Module navigation

Notes is at ⌘4 (already wired from Wave 2). Clicking opens the Notes module surface with sidebar and main content area.

3.5.2 Notes sidebar

NOTES



All notes

(12)



Recent

— FOLDERS — [+]

▼ Work

▼ Q2

Planning Meetings

Strategy Notes

Operations

▼ Personal

Reading Notes

— PURPOSES — [+]

Meeting Note (5)

Project Brief	(2)
Reading Note	(3)
Note	(4)

– TABLES (placeholder for 4b) –
(empty for Wave 4a)

[+ New note] [+ New folder]

The Purposes section provides a flat view filtered by Purpose type — useful for “show me all my Meeting Notes” without folder navigation.

The Tables section is a visual placeholder; Wave 4b populates it.

3.5.3 Notes list view

When a folder is selected, main content area shows the notes in that folder:

```
+-----+
|  Q2 Planning Meetings                               [...] [+] |
+-----+
|
|  [Search notes in this folder...]
|
|  [
|    ♦ Q2 Kickoff Meeting
|    Meeting Note · April 15 · Atlas project
|    Discussed timeline, scope, and team allocations...
|  ]
|
|  [
|    Q2 Strategy Brief
|    Project Brief · April 10 · Atlas project (★ brief)
|    Atlas Phase 1 builds the GTD core, then notes...
|  ]
|
+-----+
```

Each note card shows:

- Title
- Purpose label and date
- Project name if attached (with ★ if it's the brief)

- Two-line preview of content
- Hover affordance: open inspector, change purpose, move to folder, delete

Click → opens the note in the editor.

3.5.4 Note editor view

Full-screen editor with surrounding metadata panel:

+-----+-----+		+-----+-----+	
	← Back		⋮
			METADATA
	Q2 Kickoff Meeting		Purpose
			[Meeting Note ▼]
	## Attendees		
	- Umar (host)		Project
	- Sarah, Asghar, Jameel		Atlas
	## Discussion		★ Mark as brief
	We aligned on Phase 1 scope and timeline.		
	Decided to start with <code>[[Tasks Module]]</code> in <code>[[Wave 3a]]</code> .		Folder
			Work / Q2
	## Action items		
	- [] Send agenda to team @sarah		Tags
	- [] Block calendar for review @umar		(none yet)
			Created Apr 15
			Updated 2h ago
			— BACKLINKS —
			Mentioned in:
			• Atlas Brief
			• Q2 Planning
			— ATTACHMENTS —
			(drag files here)
+-----+-----+		+-----+-----+	

The metadata panel is collapsible (toggle to give the editor more horizontal space).

Reference rendering:

Inside the editor, `[[Tasks Module]]` renders as a styled link (subtle background, link color).

Click → navigates to that entity. The link is interactive: hover shows a small preview tooltip with the linked entity's title and first line.

Action items (task lists):

- [] markdown becomes an interactive checkbox in the editor. Clicking the box toggles state. These are inline checkboxes within the note — they DON'T create Task entities (those are explicit). They're just visual checkboxes for inline notes-style task tracking.

If a user wants real tasks linked from a note, they use `[[Task Title]]` references or use the `/task` slash command (which creates a real Task scoped to the note's project, if any).

3.5.5 Purpose templates

When creating a new note, user picks a Purpose. Each Purpose has a starting template:

Meeting Note template:

```
# [Title]

**Date**: [Today]
**Attendees**:
**Project**:

## Discussion

## Decisions

## Action items
- [ ]
```

Project Brief template:

```
# [Title]

## Overview
What is this initiative?

## Goals
What are we trying to achieve?

## Approach
How will we get there?

## Status
```

Related

-

Reading Note template:

[Title]

Source:

Author:

Date read:

Key ideas

Quotes

My thoughts

Note (default) template:

Empty — just a title and blank body.

Templates are starting structures. Users freely modify after creation. They don't enforce schemas.

3.6 Project Type display in sidebar

In the Tasks module sidebar, the Projects section is now grouped by Type:

PROJECTS

[+]

▼ Projects (3)

- Atlas
- Q2 Strategy
- Operations Q2

▼ Goals (2)

- Run a half marathon
- Save for emergency fund

▼ Habits (4)

- Meditate daily
- Read 30 min daily
- Weekly review
- Monthly finance review

Each Type group has its own collapse state (persisted in user preferences). All-empty groups hide entirely (if user has no Goals, the Goals group doesn't show).

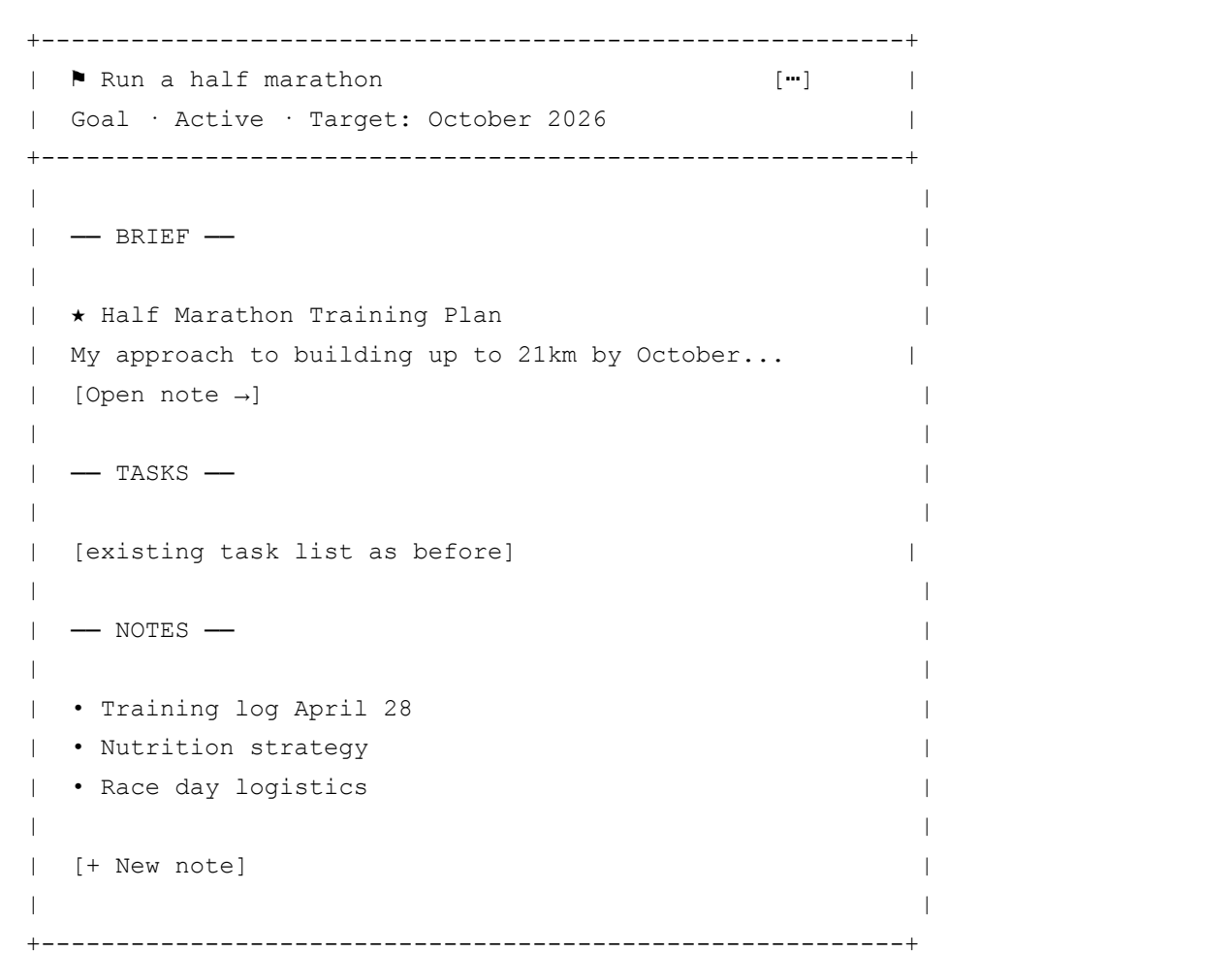
The “+” button on the section header opens a Type picker:



Click → name dialog → creates project of selected type. (Wave 4c adds richer template scaffolding; for 4a, creating a Goal just creates a Project with `type='goal'` and the user adds notes/tasks manually.)

3.7 Project view shows associated notes

In the existing project view (when user clicks a project), the layout gains a Notes section:



The brief note (if designated) appears prominently at top. Other notes listed below the tasks. This is the foundation for Wave 4c's full aggregated view; for 4a, just notes are shown.

Brief designation UX:

In the note editor's metadata panel, "★ Mark as brief" toggle. Setting it on:

- Sets `is_project_brief = true` on this note
- If another note in the same project had `is_project_brief = true`, that flag is removed (only one brief per project)
- The project view immediately reflects the new brief

3.8 Locale setting

3.8.1 Schema additions to User

```
model User {
  // existing fields...

  locale_preset    String    @default("pakistan") // 'pakistan' | 'us' | 'uk' | '

  // Granular fields (set automatically from preset, or manually if locale_preset
  currency_code    String    @default("PKR")       // ISO 4217 (PKR, USD, EUR, GBP
  currency_symbol  String    @default("Rs")
  date_format      String    @default("dd-mm-yyyy") // 'dd-mm-yyyy' | 'mm-dd-yyyy'
  number_format    String    @default("1,234.56")   // '1,234.56' | '1.234,56' | '
  time_format      String    @default("12h")        // '12h' | '24h'

  // existing fields continued...
}
```

3.8.2 Locale presets

```
const LOCALE_PRESETS = {
  pakistan: {
    name: "Pakistan",
    currency_code: "PKR",
    currency_symbol: "Rs",
    date_format: "dd-mm-yyyy",
    number_format: "1,234.56",
    time_format: "12h",
  },
}
```

```

us: {
  name: "United States",
  currency_code: "USD",
  currency_symbol: "$",
  date_format: "mm-dd-yyyy",
  number_format: "1,234.56",
  time_format: "12h",
},
uk: {
  name: "United Kingdom",
  currency_code: "GBP",
  currency_symbol: "£",
  date_format: "dd/mm/yyyy",
  number_format: "1,234.56",
  time_format: "24h",
},
custom: {
  name: "Custom",
  // Use whatever the user has set
},
}

```

3.8.3 Settings UI

In Settings → Preferences:

```

+-----+
|  Preferences  |
+-----+
|              |
|  Locale      |
|  [Pakistan ▼] |
|              |
|  Examples with current settings:  |
|  Date:       30-04-2026           |
|  Number:     1,234.56             |
|  Currency:   ₨ 1,234.56           |
|  Time:       2:30 PM              |
|              |
|  _____ |
|              |
|  [Custom...] |
|              |
+-----+

```


When user clicks "Custom...":

```
+-----+
| Custom Locale [x] |
+-----+
|
| Currency |
| Code: [PKR ▼] Symbol: [Rs ] |
|
| Date format |
| [dd-mm-yyyy ▼] |
| Example: 30-04-2026 |
|
| Number format |
| [1,234.56 ▼] |
| Example: 1,234.56 |
|
| Time format |
| ☒ 12-hour (2:30 PM) |
| ☐ 24-hour (14:30) |
|
| [Cancel] [Save] |
+-----+
```

Saving with Custom sets `locale_preset = 'custom'` and stores the granular fields. Selecting a preset later overrides custom settings.

3.8.4 Application-wide consumption

A `useLocale()` hook returns the current user's locale settings. All date/number/currency rendering across the app uses formatter functions that consume this:

```
import { useLocale, formatDate, formatNumber, formatCurrency } from '@core/local

// In components:
const locale = useLocale()
formatDate(someDate, locale) // "30-04-2026" or "04/30/2026" depending
formatNumber(1234.56, locale) // "1,234.56" or "1.234,56"
formatCurrency(1234.56, locale) // "Rs 1,234.56" or "$1,234.56"
formatTime(someDate, locale) // "2:30 PM" or "14:30"
```

Update existing date/time/number displays in:

- Today/Tomorrow/Forecast headers

- Task row dates
- Activity feed timestamps
- Audit log
- Capture timestamps
- Inspector dates

This is mostly mechanical — find existing formatters, replace with locale-aware ones.

3.9 Scheduled job runner

3.9.1 Library choice

Use `pg-boss` (Postgres-backed job queue). Reasons:

- No Redis dependency (simpler infrastructure)
- Persists in the same database as everything else
- Mature, well-maintained
- Sufficient for the scale of personal productivity (jobs in dozens per hour, not thousands per second)

3.9.2 Initial job set

```
const jobs = [
  {
    name: 'drive-sync-notes',
    schedule: '0 * * * *', // Hourly at minute 0
    handler: syncNotesToDrive,
    description: 'Backup all notes to Google Drive as Markdown files',
  },
  {
    name: 'drive-sync-tables', // Placeholder for 4b
    schedule: '0 * * * *',
    handler: noOp, // No-op until tables exist
    description: 'Backup all tables to Google Drive as CSV+JSON',
  },
  {
    name: 'session-cleanup',
    schedule: '0 3 * * *', // Daily at 3am
    handler: cleanupExpiredSessions,
    description: 'Remove expired Clerk sessions older than 30 days',
  },
];
```

```

},
{
  name: 'trash-retention',
  schedule: '0 4 * * *', // Daily at 4am
  handler: cleanupOldTrash,
  description: 'Permanently delete items in trash older than 30 days',
},
{
  name: 'attachment-cleanup',
  schedule: '0 5 * * *', // Daily at 5am
  handler: cleanupOrphanedAttachments,
  description: 'Remove orphaned attachments older than 90 days (with no parent)',
},
]

```

Each job is registered on app startup. The runner picks up due jobs and executes them.

3.9.3 Settings UI for jobs

In Settings → System (new section) → Jobs:

```

+-----+
| Scheduled Jobs |
+-----+
|
|
| [Box]
| | Drive sync - Notes [Active] |
| | Backup all notes to Google Drive as Markdown |
| | Schedule: Every hour |
| | Last run: 3:00 PM (success, 12 files synced) |
| | Next run: 4:00 PM |
| | [Run now] [Pause] |
| |
|
|
| [Box]
| | Session cleanup [Active] |
| | Remove expired sessions |
| | Schedule: Daily at 3:00 AM |
| | Last run: 3:00 AM today (success, 4 cleaned) |
| | Next run: 3:00 AM tomorrow |
| | [Run now] [Pause] |
| |
|
| ...
|
+-----+

```

Each job card shows status, schedule, last/next run, and provides Run-now and Pause/Resume controls.

Run now: triggers immediate execution outside the schedule. Useful for “I want to back up to Drive right now” without waiting for the hourly tick.

Pause/Resume: temporarily disables the schedule. Job remains registered; just doesn’t fire until resumed. Useful for travel, debugging, or if a job is misbehaving.

3.10 Drive sync of notes

3.10.1 Drive folder structure

```
Atlas/  
  Notes/  
    Meeting Note/  
      2026-04-15-q2-kickoff-meeting.md  
      2026-04-22-team-1-1.md  
    Project Brief/  
      2026-04-01-atlas-phase-1-plan.md  
      2026-04-10-q2-strategy-brief.md  
    Reading Note/  
      2026-04-12-stoicism-research.md  
    Note/  
      2026-04-30-random-thoughts.md  
      2026-04-28-meeting-prep.md
```

Folder structure mirrors the Purpose. Filenames are `YYYY-MM-DD-slug-of-title.md`. Date is the Note’s `created_at`.

If two notes have the same date and slug (rare but possible), append a suffix: `2026-04-15-q2-kickoff-meeting-2.md`.

3.10.2 File contents

Each `.md` file contains frontmatter + body:

```
---  
atlas_id: 019dd9d4-a8c5-7000-8000-000000000000  
title: Q2 Kickoff Meeting  
purpose: meeting_note  
project: Atlas  
folder: Work/Q2
```

```
created_at: 2026-04-15T10:30:00Z
updated_at: 2026-04-22T14:15:00Z
tags: [q2, planning]
exported_at: 2026-04-30T15:00:00Z
---
```

```
# Q2 Kickoff Meeting
```

```
## Attendees
```

```
- Umar (host)
- Sarah, Asghar, Jameel
```

```
## Discussion
```

```
We aligned on Phase 1 scope and timeline.
Decided to start with [[Tasks Module]] in [[Wave 3a]].
```

```
## Action items
```

```
- [ ] Send agenda to team @sarah
- [ ] Block calendar for review @umar
```

References (`[[X]]` , `@X` , `#X`) are preserved as-is in the markdown — they're meaningful as text even if the linked entities aren't accessible from outside Atlas.

3.10.3 Sync logic

For each note:

1. Compute target Drive path from Purpose + filename
2. Generate markdown content (frontmatter + body_markdown)
3. If `drive_file_id` is set, update existing Drive file
4. If not set, create new Drive file in the appropriate folder (creating folders as needed)
5. Store the returned `drive_file_id` and `drive_synced_at`
6. On error, store `drive_sync_error` and continue with next note

Overwrite, no versioning: every sync overwrites the same file. Drive itself maintains version history (Google Drive automatically keeps revisions for ~30 days). Atlas doesn't try to manage Drive versioning.

Handling renames/purpose changes:

If a note's title changes, its filename changes. Sync logic:

1. If new filename != old filename, delete old Drive file before creating new one

2. Track old filename via a hidden field if needed, or query Drive for the file by `atlas_id` in `frontmatter`

Same for purpose changes (file moves to different folder).

For simplicity in v1: when title or purpose changes, delete the old Drive file (if `drive_file_id` exists) and create a fresh one. The hourly job handles this naturally on next run.

3.10.4 Folder lifecycle in Drive

Folders are created on-demand. If user has no notes with `purpose = 'reading_note'`, no `Reading Note/` folder exists in Drive. When the first reading note syncs, the folder is created.

Empty folders aren't cleaned up (user might create another note of that purpose later).

3.11 Audit log additions

New audit actions:

- `note_created`
- `note_updated`
- `note_deleted`
- `note_restored`
- `note_purpose_changed`
- `note_project_changed`
- `note_brief_designated`
- `note_brief_undesignated`
- `notes_folder_created`
- `notes_folder_renamed`
- `notes_folder_moved`
- `notes_folder_deleted`
- `link_created` (when reference resolved during note save)
- `link_removed`

- project_type_changed
- project_status_changed
- locale_changed
- job_run_completed (system action, not user)
- job_paused / job_resumed

These appear in the relevant entity's Activity feed where applicable.

4. tRPC procedures

```
// Notes
notes.create({ title, body_json?, folder_id?, project_id?, purpose? })
notes.list({ folder_id?, project_id?, purpose? }) → Note[]
notes.byId({ id }) → Note with backlinks resolved
notes.update({ id, title?, body_json?, purpose?, folder_id?, project_id? })
notes.delete({ id })
notes.designateBrief({ note_id, project_id }) // Marks note as project's brief
notes.undesignateBrief({ note_id })

// Notes folders
notesFolder.create({ name, parent_folder_id? })
notesFolder.list() → tree structure
notesFolder.rename({ id, new_name })
notesFolder.move({ id, new_parent_folder_id? }) // null = root
notesFolder.delete({ id }) // Cascades soft-delete to notes within

// Project enhancements
projects.update({ id, type?, status?, target_date?, ... }) // Existing procedure

// Search
search.notes({ query, folder_id?, project_id?, purpose? }) // Full-text search

// Links
links.outbound({ source_type, source_id }) → Link[]
links.inbound({ target_type, target_id }) → Link[] // Backlinks

// Locale
user.updateLocale({ preset?, currency_code?, currency_symbol?, date_format?, numb

// Jobs
jobs.list() → JobInfo[]
```

```
jobs.runNow({ job_name })
jobs.pause({ job_name })
jobs.resume({ job_name })
```

5. File structure additions

```
/atlas
/app
  /(app)
    /notes
      /page.tsx (Notes module home, folder tree)
      /[noteId]/page.tsx (Note editor view)
      /folder/[folderId]/page.tsx (Folder contents)
      /purpose/[purpose]/page.tsx (Notes filtered by purpose)
    /settings
      /preferences/page.tsx (Locale settings)
      /system/page.tsx (System / jobs management)
/components
  /notes
    note-editor.tsx (TipTap editor wrapper)
    note-metadata-panel.tsx (Right sidebar in editor)
    note-list-item.tsx (Cards in folder view)
    note-card.tsx (Compact card)
    reference-picker.tsx (Dropdown for [, #, @ triggers)
    purpose-template-picker.tsx (Choosing template on creation)
    backlinks-panel.tsx (Inbound references display)
    brief-toggle.tsx (Mark/unmark as project brief)
  /notes/sidebar
    notes-folder-tree.tsx
    notes-purpose-list.tsx
  /projects
    project-type-selector.tsx
    project-status-selector.tsx
    project-target-date-picker.tsx
    project-notes-section.tsx (Notes section in project view)
    project-brief-display.tsx
  /sidebar
    projects-by-type.tsx (Grouped sidebar display)
  /settings
    locale-preset-selector.tsx
    locale-custom-form.tsx
    locale-preview.tsx (Shows examples with current settings)
    jobs-management.tsx (Job list with controls)
    job-card.tsx
```



```

/core
  /locale
    formatters.ts          (formatDate, formatNumber, formatCurr
    presets.ts             (LOCALE_PRESETS constants)
    hooks.ts               (useLocale)
  /editor
    tiptap-config.ts       (Editor extensions and configuration)
    reference-extension.ts  (Custom TipTap extension for [[, #, @
    markdown-export.ts     (TipTap JSON → Markdown)
    markdown-import.ts     (Markdown → TipTap JSON)
    text-extraction.ts     (TipTap JSON → plain text for search)
  /notes
    service.ts             (CRUD with link resolution)
    drive-sync.ts          (Sync logic for notes → Drive)
    filename.ts            (Generate safe filenames)
  /links
    resolver.ts            (Reference text → entity ID)
    service.ts             (Link table operations)
  /jobs
    registry.ts            (Job definitions)
    runner.ts              (pg-boss configuration and registrati
    handlers/
      sync-notes-to-drive.ts
      sync-tables-to-drive.ts (Placeholder for 4b)
      cleanup-sessions.ts
      cleanup-trash.ts
      cleanup-attachments.ts
  /server
    /routers
      notes.ts
      notesFolder.ts
      links.ts
      jobs.ts
      user.ts              (Locale additions)
      projects.ts          (Type, status, target_date additions)
      search.ts            (Notes search)

```

6. Verification (Definition of Done)

Notes core

1. Open Notes module via ⌘4
2. Create a new note → editor opens with cursor in title

3. Type title, Tab to body → start writing
4. Type # at start of line → becomes heading
5. Type ****bold**** → renders as bold inline
6. Type [[→ reference picker opens with searchable entity list
7. Pick a reference → renders as styled link in editor
8. Type # → tag picker opens (existing tags only, no auto-create)
9. Type @ → context/person picker opens
10. Auto-save fires ~1 second after typing stops
11. Cmd+S triggers immediate save
12. Refresh page → note persists with all content
13. Click [[reference]] → navigates to linked entity
14. Hover [[reference]] → tooltip shows entity preview

Folders

15. Create a folder in Notes module → appears in sidebar
16. Move a note into the folder → note relocates correctly
17. Create sub-folder (folder within folder) → nests correctly
18. Try to nest beyond 5 levels deep → blocked
19. Rename folder → reflects everywhere
20. Delete folder → notes within soft-delete (cascade)

Purposes

21. Create new note → Purpose picker offers Note, Meeting Note, Project Brief, Reading Note
22. Pick "Meeting Note" → editor opens with Meeting Note template
23. Pick "Project Brief" → editor opens with Project Brief template
24. Change Purpose of existing note via metadata panel → template doesn't reapply (content preserved)

- 25. Sidebar Purposes section shows count for each purpose
- 26. Click "Meeting Note" in Purposes → list filters to all meeting notes

Project enhancement

- 27. Existing projects show Type = "project" by default
- 28. Create new Project → Type picker offers Project, Goal, Habit
- 29. Create a Goal "Run a half marathon" → appears under Goals group in sidebar
- 30. Create a Habit "Meditate daily" → appears under Habits group
- 31. Sidebar groups (Projects, Goals, Habits) show counts and can collapse independently
- 32. Empty groups hide entirely
- 33. Set status = "completed" on a project → reflects in UI (e.g., dimmed or moved to Completed)
- 34. Set target_date on a goal → displayed in project header

Project-Note attachment

- 35. In note editor metadata, set Project = some project → note attaches
- 36. Open the project → note appears in Notes section
- 37. Toggle "Mark as brief" on a note → it becomes the project's brief
- 38. Try to mark another note as brief in the same project → previous brief is unmarked automatically
- 39. Project view shows brief prominently above tasks
- 40. Detach note from project (set Project = none) → no longer appears in project view
- 41. Note still exists in Notes module folder

Cross-module references and backlinks

- 42. Create note A with `[[Note B]]` → Link row created (resolved if Note B exists)
- 43. Open Note B → backlinks panel shows "Mentioned in: Note A"
- 44. Rename Note B → backlinks still work (using IDs, not titles)
- 45. Reference a Task from a Note via `[[Task Title]]` → Link created with `target_type='task'`

- 46. Create note with # reference → existing tag attached, OR (per refinement v2) suggested for creation if not existing — does NOT auto-create
- 47. Create note with @person → person picker shows existing contacts/people; once People module ships (Wave 6), references resolve

Locale settings

- 48. Settings → Preferences → Locale shows presets dropdown with Pakistan as default
- 49. Change to United States → date format changes to mm-dd-yyyy throughout app, currency symbol to \$
- 50. Change to Custom → custom form opens with all granular fields
- 51. Set custom values → preview updates live
- 52. Save → locale_preset = 'custom', granular fields persist
- 53. Dates in Tasks module (Today header, task rows) reflect new format
- 54. Numbers in any displayed context (estimated time, etc.) reflect new format
- 55. Time displays (audit log, activity feed) reflect 12h or 24h selection

Drive sync

- 56. Settings → System → Jobs shows Drive sync — Notes job
- 57. Click "Run now" → job triggers immediately
- 58. Check Google Drive → Atlas/Notes/[purpose folders]/ created with .md files
- 59. Each file has frontmatter (atlas_id, title, purpose, etc.) + body
- 60. Filename format: YYYY-MM-DD-slug.md
- 61. Edit a note in Atlas → wait for next hourly sync (or run now) → Drive file updates with new content
- 62. Change a note's Purpose → file moves to new purpose folder on next sync (old file deleted)
- 63. Delete a note in Atlas → on next sync, Drive file is deleted (or moved to trash; verify behavior)
- 64. Pause the Drive sync job → no syncs occur until resumed
- 65. Resume → syncs resume on next scheduled tick

Scheduled jobs UI

- 66. Settings → System shows job list
- 67. Each job card shows schedule, last run, status, next run
- 68. Run-now button works for any job
- 69. Pause/Resume button changes job state
- 70. Last run shows success/failure with timestamp
- 71. Failed jobs show error message inline

Editor edge cases

- 72. Paste rich text from external source → preserves basic formatting (bold, italic, lists)
- 73. Paste image → uploaded as attachment (using Media wave infrastructure)
- 74. Paste URL with text selected → creates hyperlink
- 75. Code block with language → syntax highlighting renders
- 76. Long note (10,000+ words) → editor remains responsive
- 77. Save during typing (mid-word) → resumes correctly without losing content

Backwards compatibility

- 78. All existing tasks, projects, captures, attachments work as before
- 79. Existing references (from Wave 3a parser) get backfilled into Link table on first save
- 80. No regression in any verification step from prior waves

When all 80 verification steps pass, Wave 4a is complete.

7. Rules of engagement

7.1 Capture before perfection in the editor

The TipTap editor will have rough edges. Some markdown shortcuts will misfire occasionally. Some pastes won't render perfectly. That's acceptable. What's NOT acceptable: data loss, autosave failures, or notes becoming inaccessible.

Prioritize:

1. Notes always save (even mid-typing if connection blips)
2. Saved notes always render correctly when reopened
3. Drive sync never overwrites with corrupt content

Polish like “perfect markdown shortcut behavior in every edge case” can come in 4c.

7.2 References use IDs, not titles

The Link table stores entity IDs. This means renaming a referenced entity doesn't break the link. The reference text in the note body should also store the resolved ID alongside the display title — TipTap's reference extension uses a node attribute for the ID:

```
{
  type: "reference",
  attrs: {
    target_type: "note",
    target_id: "019dd9d4-...",
    display_text: "Q2 Planning" // Original title; updates reactively if target
  }
}
```

When the target is renamed, the display text updates on next render (queried fresh). The link stays valid.

7.3 Drive sync is one-way always

Atlas is the source of truth. Drive is backup. Never read FROM Drive to update Atlas content. Even if a Drive file is edited externally, Atlas's hourly sync overwrites it with Atlas's current state. This is intentional — preventing two-way sync complexity.

If user edits a markdown file in Drive expecting changes to appear in Atlas, they'll be confused. The frontmatter in synced files should include a comment:

```
# Generated by Atlas. Edits to this file will be overwritten on next sync.
# To make changes, edit in Atlas at: [URL]
```

(Actually — don't include the URL since domains can change. Just the comment.)

7.4 Locale propagation must be thorough

The Locale setting is only useful if EVERYTHING respects it. After implementing the formatters and useLocale hook, audit the entire codebase for hardcoded date/number/currency rendering and migrate to locale-aware formatters.

Common places to check:

- Task row dates
- Today/Tomorrow/Forecast headers
- Activity feed timestamps
- Audit log
- Capture parse log
- Inspector dates
- Project target dates
- Estimated time displays (number formatting)

If you find a place that's still using hardcoded format, fix it.

7.5 The job runner is for personal scale

Don't over-engineer the job runner. pg-boss with its defaults is fine. We're talking dozens of jobs per hour at most. Don't add Redis, don't shard workers, don't optimize for throughput. Make it work reliably for a single user (or a small handful of family/friends) and that's the bar.

7.6 Project Type changes have implications

When a user changes a Project's Type (e.g., from Project to Goal):

- Audit log entry created
- Sidebar grouping updates immediately
- No data is lost — same entity, just relabeled
- No side effects on tasks/notes attached to the project

Don't try to be smart ("changing to Habit means we should make all tasks recurring") — that's surprising behavior. Type is a label; user manages the underlying content explicitly.

7.7 Backlinks query must be performant

The Link table will grow. The `(target_type, target_id)` index makes backlink queries fast, but the panel renders on every note open. Cache aggressively (TanStack Query) and only invalidate when the relevant note's body changes.

For really large knowledge bases (1000+ notes with many cross-refs), consider materialized backlink counts. Not needed for v1.

8. Recommended Build Sequence

The work is grouped into phases. Don't try to parallelize across phases; dependencies exist.

Phase 1: Schema and infrastructure (3-4 days)

1. Notes table, NotesFolder table, Link table — Prisma migrations
2. Project enhancement fields (type, status, target_date) — migration with safe defaults
3. Locale fields on User — migration with Pakistan defaults
4. pg-boss setup and basic job registration

Phase 2: Locale formatters (2-3 days)

5. Formatters: formatDate, formatNumber, formatCurrency, formatTime
6. useLocale hook
7. Locale settings UI (presets + custom form)
8. Migrate existing date/number displays to locale-aware formatters

Phase 3: TipTap editor (5-7 days)

9. Editor configuration and basic extensions (StarterKit, markdown shortcuts)
10. Custom reference extension ([[, #, @ handling)
11. Reference picker component
12. Markdown export/import utilities
13. Plain text extraction for search
14. Auto-save and dirty state management

Phase 4: Notes module UI (4-5 days)

15. Notes module page, sidebar, folder tree
16. Note list view (folder contents)
17. Note editor view with metadata panel
18. Purpose templates (Meeting Note, Project Brief, Reading Note)
19. Search across notes

20. Backlinks panel

Phase 5: Project enhancement (2-3 days)

21. Project Type selector and status selector

22. Sidebar grouping by Type

23. Project view shows Notes section

24. Brief designation (Mark/unmark)

Phase 6: Link resolution and backfill (2-3 days)

25. Link resolver service

26. Hook into note save: parse references, create/update Link rows

27. Hook into existing entity saves (tasks): same parsing, Link rows

28. Backfill: scan all existing tasks for references, populate Link table

Phase 7: Drive sync (3-4 days)

29. Drive sync handler for notes

30. Filename generation

31. Frontmatter generation

32. Folder creation in Drive

33. Update/delete existing files

34. Wire up to job runner (hourly + run-now)

Phase 8: Jobs management UI (2 days)

35. Settings → System → Jobs page

36. Job cards with run-now, pause/resume

37. Last run / next run display

Phase 9: Verification

38. All 80 verification steps

This sequence builds infrastructure first (schemas, locale, jobs), then the editor (the hardest piece), then the UI layers, then the integrations (links, Drive sync), then the management surfaces.

9. What is NOT in Wave 4a

Wave 4b territory:

- Tables (entity, columns, editing, views)
- CSV+JSON Drive backup for tables
- Side panel display when referenced via [[Table]]

Wave 4c territory:

- Unified Project view aggregating notes, tasks, tables, calendar events, journal entries
- "+ New Goal/Habit" with template scaffolding (auto-create note + tasks + table pre-linked)
- Note versioning / history
- Public sharing of notes
- Embed types in notes (video, images as embeds beyond simple attachments, code blocks with execution)
- Multi-select column type for tables
- Formula column type
- Relation column type (table-to-table links)
- CSV import for tables

Phase 2 territory:

- Visual knowledge graph view
- Collaborative editing of notes
- Note templates editor (user-defined templates)
- Per-note locale override (shouldn't be needed; global is enough)
- Real-time Drive sync (vs scheduled hourly)

Wave 5+ territory:

- Calendar two-way sync (Wave 5)
- People module (Wave 6) — until then, @person references resolve to contacts (existing)

or stub for later

- Journals module (Wave 7)
- Vault module (Phase 2)

If you find yourself building any of these, stop.

10. Final note

Wave 4a is the largest single wave in Phase 1 by scope. The TipTap editor alone is most of a week. Drive sync is multiple days. The infrastructure pieces (locale, jobs, links) compound.

The temptation will be to ship something less polished to “save time.” Resist this. Notes is the second pillar of Atlas after Tasks; it has to feel as good as Tasks does. A janky editor or unreliable Drive sync poisons trust in the entire app.

Build it carefully. Verify thoroughly. The 4-5 week estimate is realistic; don’t compress.

The result is meaningful: Atlas becomes a system where any sustained initiative — Project, Goal, Habit — has the Notes layer for the “what and why” alongside the Tasks layer for the “how.” The composition pattern that Wave 4c activates is foundational to making Atlas feel like a personal command center, not just a task manager with extras.

Begin with section 8, Phase 1.